

File system data structures must persist.

One major challenge faced by file systems is how to update persistent data structures despite the presence of a power loss or system crash.

Specifically, what happens if, right in the middle of updating on-disk structures, someone trips over the power cord and the machine loses power? Or the operating system encounters a bug & crashes?

This problem is known as crash-consistency problem.

Imagine you have to update two on-disk structures, A & B, to complete a particular operation. Because the disk only services a single request at a time, one of these requests will reach the disk first (either A or B).

If the system crashes or loses power after one write completes, the on-disk structure will be left in an inconsistent state.

THE CRUX: HOW TO UPDATE THE DISK DESPITE CRASHES

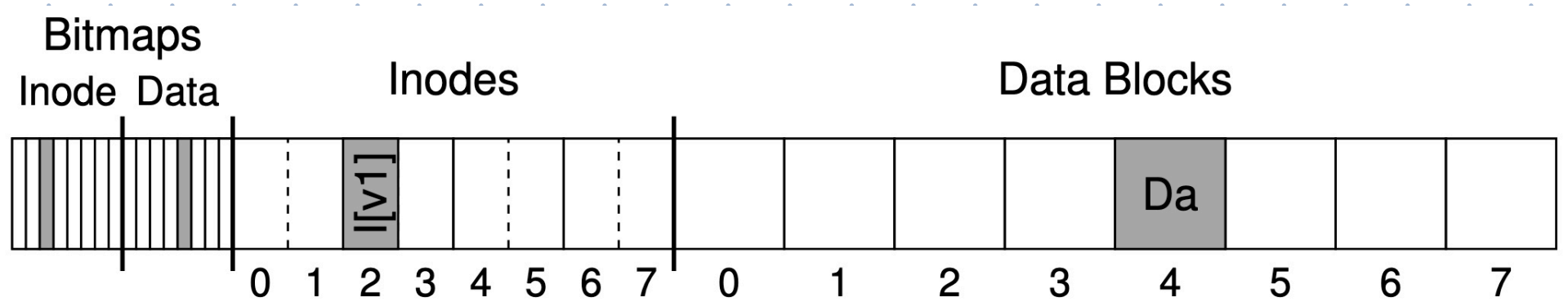
The system may crash or lose power between any two writes, and thus the on-disk state may only partially get updated. After the crash, the system boots and wishes to mount the file system again (in order to access files and such). Given that crashes can occur at arbitrary points in time, how do we ensure the file system keeps the on-disk image in a reasonable state?

A Detailed Example

We will use a workload that updates on-disk structures in some way. Assume the workload is: the append of a single data block to an existing file.

The append is accomplished by opening the file, calling `lseek()` to move the offset to end of the file, and then issuing a single 4KB write to the file before closing it.

Lets assume we are using a standard simple file system structures on the disk, which includes an inode bitmap (8 bits), a data bitmap (8 bits), inodes (8 total) & data blocks (8 total):



A single inode $I[V_1]$ ^{→ 1st version of inode. It will be updated soon by the workload} and a single data block Da is allocated, which are marked in inode & data bitmaps respectively.

Inside of $I[V_1]$:

```
owner      : remzi
permissions : read-write
size       : 1
pointer    : 4
pointer    : null
pointer    : null
pointer    : null
```

When we append to the file, we are adding new data block to it, and thus must update three on-disk structures:

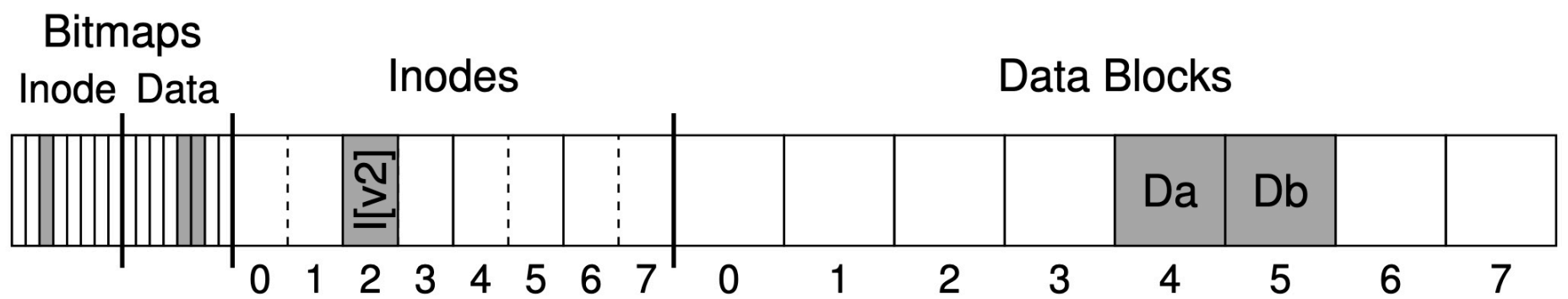
1. The inode $I[V_2]$
2. The new data block Db
3. A new version of data bitmap $B[V_2]$ to indicate new data block Db has been allocated.

Inside of the updated inode $I[V_2]$:

```
owner      : remzi
permissions : read-write
size       : 2
pointer    : 4
pointer    : 5
pointer    : null
```


pointer : null

The on-disk image file system looks like:



To achieve this, the file system must perform three separate writes to the disk, one for inode $I[v_2]$, bitmap $B[v_2]$, and data block Db .

These writes don't happen immediately when the user issues `write()` system call; rather, they will sit in page cache or buffer cache for some time first, & then the file system will issue the requisite write requests to the disk.

Unfortunately, a crash may occur & thus interfere with these updates to the disk.

Crash Scenarios

Imagine only a single write succeeds; thus there are three possible outcomes:

- **Just the data block (Db) is written to disk.** In this case, the data is on disk, but there is no inode that points to it and no bitmap that even says the block is allocated. Thus, it is as if the write never occurred. This case is not a problem at all, from the perspective of file-system crash consistency¹.
- **Just the updated inode ($I[v_2]$) is written to disk.** In this case, the inode points to the disk address (5) where Db was about to be written, but Db has not yet been written there. Thus, if we trust that pointer, we will read **garbage** data from the disk (the old contents of disk address 5).

Further, we have a new problem, which we call a **file-system inconsistency**. The on-disk bitmap is telling us that data block 5 has not been allocated, but the inode is saying that it has. The disagreement between the bitmap and the inode is an inconsistency in the data structures of the file system; to use the file system, we must somehow resolve this problem (more on that below).

- **Just the updated bitmap ($B[v2]$) is written to disk.** In this case, the bitmap indicates that block 5 is allocated, but there is no inode that points to it. Thus the file system is inconsistent again; if left unresolved, this write would result in a **space leak**, as block 5 would never be used by the file system.

Now, imagine two writes succeed; thus three scenarios are possible:

- **The inode ($I[v2]$) and bitmap ($B[v2]$) are written to disk, but not data (Db).** In this case, the file system metadata is completely consistent: the inode has a pointer to block 5, the bitmap indicates that 5 is in use, and thus everything looks OK from the perspective of the file system's metadata. But there is one problem: 5 has garbage in it again.
- **The inode ($I[v2]$) and the data block (Db) are written, but not the bitmap ($B[v2]$).** In this case, we have the inode pointing to the correct data on disk, but again have an inconsistency between the inode and the old version of the bitmap ($B1$). Thus, we once again need to resolve the problem before using the file system.
- **The bitmap ($B[v2]$) and data block (Db) are written, but not the inode ($I[v2]$).** In this case, we again have an inconsistency between the inode and the data bitmap. However, even though the block was written and the bitmap indicates its usage, we have no idea which file it belongs to, as no inode points to the file.

The Crash Consistency Problems

Ideally, we want to move from one consistent state to another atomically.

Unfortunately, we can't do this easily because the disk commits one write at a time, and crash or power loss may occur b/w these updates.

We call this general problem crash-consistency problem or consistent-update problem.

Solution #1: The File System Checker

Early file systems took a simple approach to crash consistency. Basically, they decided to let inconsistencies happen & then fix them later.

`fsck` is a UNIX tool for finding such inconsistencies & repairing them.

Such an approach can't fix all problems; consider, for example, the case where file system looks consistent but inode points to garbage data.

The only real goal is to make sure the file system metadata is internally consistent.

`fsck` operates in a number of phases. It is run before the file system is mounted & made available.

`fsck` assumes no other file system activity is on-going while it is running. Basic summary of what `fsck` does:

- **Superblock:** `fsck` first checks if the superblock looks reasonable, mostly doing sanity checks such as making sure the file system size is greater than the number of blocks that have been allocated. Usually the goal of these sanity checks is to find a suspect (corrupt) superblock; in this case, the system (or administrator) may decide to use an alternate copy of the superblock.
- **Free blocks:** Next, `fsck` scans the inodes, indirect blocks, double indirect blocks, etc., to build an understanding of which blocks are currently allocated within the file system. It uses this knowledge to produce a correct version of the allocation bitmaps; thus, if there is any inconsistency between bitmaps and inodes, it is resolved by trusting the information within the inodes. The same type of check is performed for all the inodes, making sure that all inodes that look like they are in use are marked as such in the inode bitmaps.
- **Inode state:** Each inode is checked for corruption or other problems. For example, `fsck` makes sure that each allocated inode has a valid type field (e.g., regular file, directory, symbolic link, etc.). If there are problems with the inode fields that are not easily fixed, the inode is considered suspect and cleared by `fsck`; the inode bitmap is correspondingly updated.
- **Inode links:** `fsck` also verifies the link count of each allocated inode. As you may recall, the link count indicates the number of dif-

ferent directories that contain a reference (i.e., a link) to this particular file. To verify the link count, `fsck` scans through the entire directory tree, starting at the root directory, and builds its own link counts for every file and directory in the file system. If there is a mismatch between the newly-calculated count and that found within an inode, corrective action must be taken, usually by fixing the count within the inode. If an allocated inode is discovered but no directory refers to it, it is moved to the `lost+found` directory.

- **Duplicates:** `fsck` also checks for duplicate pointers, i.e., cases where two different inodes refer to the same block. If one inode is obviously bad, it may be cleared. Alternately, the pointed-to block could be copied, thus giving each inode its own copy as desired.
- **Bad blocks:** A check for bad block pointers is also performed while scanning through the list of all pointers. A pointer is considered “bad” if it obviously points to something outside its valid range, e.g., it has an address that refers to a block greater than the partition size. In this case, `fsck` can’t do anything too intelligent; it just removes (clears) the pointer from the inode or indirect block.
- **Directory checks:** `fsck` does not understand the contents of user files; however, directories hold specifically formatted information created by the file system itself. Thus, `fsck` performs additional integrity checks on the contents of each directory, making sure that “.” and “..” are the first entries, that each inode referred to in a directory entry is allocated, and ensuring that no directory is linked to more than once in the entire hierarchy.

↳ Each directory has only one parent
↳ Directories can't have hard links

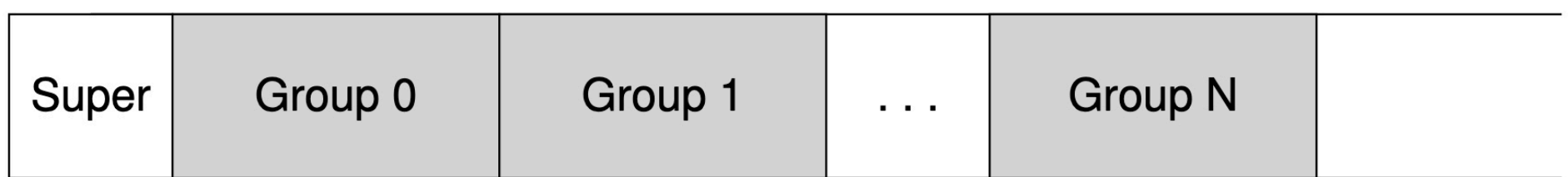
`fsck` is too slow. Performance of `fsck`, as disks grew in capacity and RAID's grew in popularity, became prohibitive.

Solution #2: Journaling (or Write-Ahead Logging)

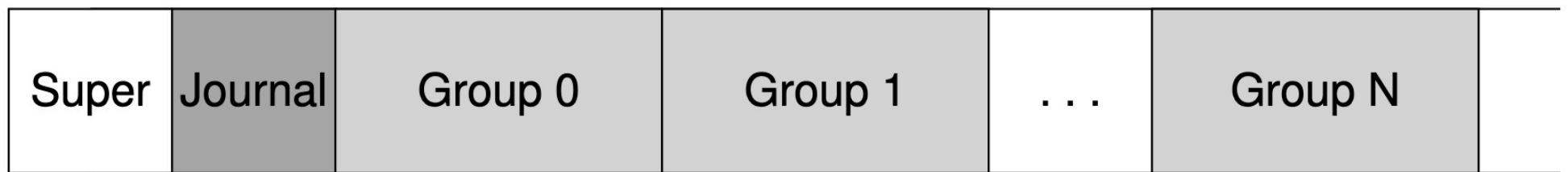
The first file system to do this was Cedar, though many modern file systems use the idea, including Linux `ext3`, `ext4`, `reiserfs`, IBM's `JFS`, SG's `XFS`, and Windows `NTFS`.

The basic idea is as follows. When updating the disk, before overwriting the structures in place, first write down a little note (somewhere else on the disk, in a well-known location) describing what you are about to do. Writing this note is the “write ahead” part, and we write it to a structure that we organize as a “log”; hence, write-ahead logging. By writing the note to disk, you are guaranteeing that if a crash takes place during the update (overwrite) of the structures you are updating, you can go back and look at the note you made and try again; thus, you will know exactly what to fix (and how to fix it) after a crash, instead of having to scan the entire disk. By design, journaling thus adds a bit of work during updates to greatly reduce the amount of work required during recovery.

This book describes how Linux ext3 incorporates journaling. Except a Journal block, on-disk structures are identical to Linux ext2, which looks like:



Assuming the Journal is placed within the same file system image (though sometimes it is placed on a separate device or as a file within the file system), an ext3 looks like:



Data Journaling

Say we have our canonical update again, where we wish to write the inode (I[v2]), bitmap (B[v2]), and data block (Db) to disk again. Before writing them to their final disk locations, we are now first going to write them to the log (a.k.a. journal). This is what this will look like in the log:



You can see we have written five blocks here. The transaction begin (TxB) tells us about this update, including information about the pending update to the file system (e.g., the final addresses of the blocks I[v2], B[v2], and Db), and some kind of **transaction identifier (TID)**. The middle three blocks just contain the exact contents of the blocks themselves; this is known as **physical logging** as we are putting the exact physical contents of the update in the journal (an alternate idea, **logical logging**, puts a more compact logical representation of the update in the journal, e.g., "this update wishes to append data block Db to file X", which is a little more complex but can save space in the log and perhaps improve performance). The final block (TxE) is a marker of the end of this transaction, and will also contain the TID.

Once this transaction is safely on disk, we are ready to overwrite the old structures in the file system; this process is called **checkpointing**. Thus, to **checkpoint** the file system (i.e., bring it up to date with the pending update in the journal), we issue the writes I[v2], B[v2], and Db to their disk locations as seen above; if these writes complete successfully, we have successfully checkpointed the file system and are basically done. Thus, our initial sequence of operations:

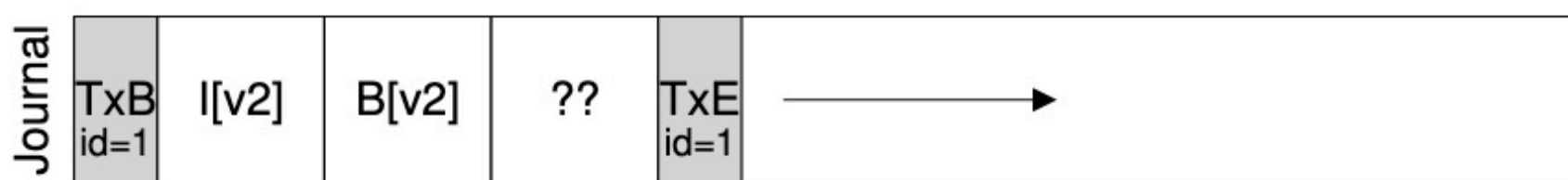
Thus, our initial sequence of operations:

1. **Journal write:** Write the transaction, including a transaction-begin block, all pending data and metadata updates, and a transaction-end block, to the log; wait for these writes to complete.
2. **Checkpoint:** Write the pending metadata and data updates to their final locations in the file system.

In our example, we would write TxB, I[v2], B[v2], Db, and TxE to the journal first. When these writes complete, we would complete the update by checkpointing I[v2], B[v2], and Db, to their final locations on disk.

Things get a little trickier when a crash occurs during the writes to the journal. Here, we are trying to write the set of blocks in the transaction (e.g., TxB, I[v2], B[v2], Db, TxE) to disk. One simple way to do this would be to issue each one at a time, waiting for each to complete, and then issuing the next. However, this is slow. Ideally, we'd like to issue

all five block writes at once, as this would turn five writes into a single sequential write and thus be faster. However, this is unsafe, for the following reason: given such a big write, the disk internally may perform scheduling and complete small pieces of the big write in any order. Thus, the disk internally may (1) write TxB, I[v2], B[v2], and TxE and only later (2) write Db. Unfortunately, if the disk loses power between (1) and (2), this is what ends up on disk:

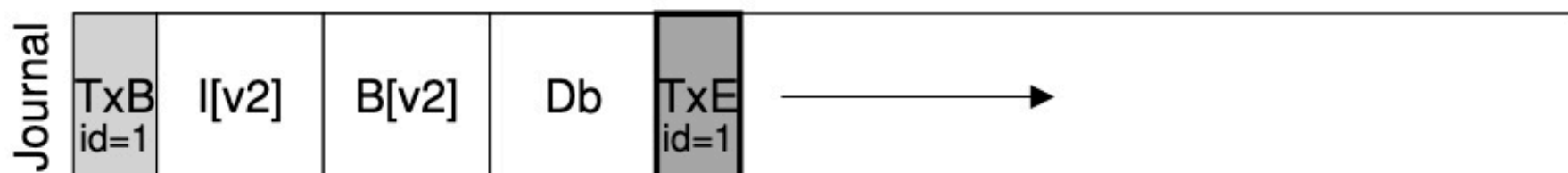


Why is this a problem? Well, the transaction looks like a valid transaction (it has a begin and an end with matching sequence numbers). Further, the file system can't look at that fourth block and know it is wrong; after all, it is arbitrary user data. Thus, if the system now reboots and runs recovery, it will replay this transaction, and ignorantly copy the contents of the garbage block '??' to the location where Db is supposed to live. This is bad for arbitrary user data in a file; it is much worse if it happens to a critical piece of file system, such as the superblock, which could render the file system unmountable.

To avoid this problem, the file system issues the transactional write in two steps. First, it writes all blocks except the TxE block to the journal, issuing these writes all at once. When these writes complete, the journal will look something like this (assuming our append workload again):



When those writes complete, the file system issues the write of the TxE block, thus leaving the journal in this final, safe state:



An important aspect of this process is the atomicity guarantee provided by the disk. It turns out that the disk guarantees that any 512-byte write will either happen or not (and never be half-written); thus, to make sure the write of TxE is atomic, one should make it a single 512-byte block. Thus, our current protocol to update the file system, with each of its three phases labeled:

1. **Journal write:** Write the contents of the transaction (including TxB, metadata, and data) to the log; wait for these writes to complete.
2. **Journal commit:** Write the transaction commit block (containing TxE) to the log; wait for write to complete; transaction is said to be **committed**.
3. **Checkpoint:** Write the contents of the update (metadata and data) to their final on-disk locations.

ASIDE: FORCING WRITES TO DISK

To enforce ordering between two disk writes, modern file systems have to take a few extra precautions. In olden times, forcing ordering between two writes, A and B , was easy: just issue the write of A to the disk, wait for the disk to interrupt the OS when the write is complete, and then issue the write of B .

Things got slightly more complex due to the increased use of write caches within disks. With write buffering enabled (sometimes called **immediate reporting**), a disk will inform the OS the write is complete when it simply has been placed in the disk's memory cache, and has not yet reached disk. If the OS then issues a subsequent write, it is not guaranteed to reach the disk after previous writes; thus ordering between writes is not preserved. One solution is to disable write buffering. However, more modern systems take extra precautions and issue explicit **write barriers**; such a barrier, when it completes, guarantees that all writes issued before the barrier will reach disk before any writes issued after the barrier.

All of this machinery requires a great deal of trust in the correct operation of the disk. Unfortunately, recent research shows that some disk manufacturers, in an effort to deliver "higher performing" disks, explicitly ignore write-barrier requests, thus making the disks seemingly run faster but at the risk of incorrect operation [C+13, R+11]. As Kahan said, the fast almost always beats out the slow, even if the fast is wrong.

ASIDE: OPTIMIZING LOG WRITES

You may have noticed a particular inefficiency of writing to the log. Namely, the file system first has to write out the transaction-begin block and contents of the transaction; only after these writes complete can the file system send the transaction-end block to disk. The performance impact is clear, if you think about how a disk works: usually an extra rotation is incurred (think about why). → By the time the disk finished step 1, step 2 request was issued. The place to write TxE had long passed the disk head, which is why a full rotation was required.

One of our former graduate students, Vijayan Prabhakaran, had a simple idea to fix this problem [P+05]. When writing a transaction to the journal,

include a checksum of the contents of the journal in the begin and end blocks. Doing so enables the file system to write the entire transaction at once, without incurring a wait; if, during recovery, the file system sees a mismatch in the computed checksum versus the stored checksum in the transaction, it can conclude that a crash occurred during the write of the transaction and thus discard the file-system update. Thus, with a small tweak in the write protocol and recovery system, a file system can achieve faster common-case performance; on top of that, the system is slightly more reliable, as any reads from the journal are now protected by a checksum.

Essentially, a number is computed based on content $b/w \text{ Tx B} \times \text{Tx E}$. This number is stored in $\text{Tx E} \times \text{Tx B}$.

During Recovery, this number is recomputed & if it is same as stored. Then, this transaction is valid.

Recovery

Let's now understand how a file system can use the contents of the journal to **recover** from a crash. A crash may happen at any time during this sequence of updates. If the crash happens before the transaction is written safely to the log (i.e., before Step 2 above completes), then our job is easy: the pending update is simply skipped. If the crash happens after the transaction has committed to the log, but before the checkpoint is complete, the file system can **recover** the update as follows. When the system boots, the file system recovery process will scan the log and look for transactions that have committed to the disk; these transactions are thus **replayed** (in order), with the file system again attempting to write out the blocks in the transaction to their final on-disk locations. This form of logging is one of the simplest forms there is, and is called **redo logging**. By recovering the committed transactions in the journal, the file system ensures that the on-disk structures are consistent, and thus can proceed by mounting the file system and readying itself for new requests.

Note that it is fine for a crash to happen at any point during checkpointing, even after some of the updates to the final locations of the blocks have completed. In the worst case, some of these updates are simply performed again during recovery. Because recovery is a rare operation (only taking place after an unexpected system crash), a few redundant writes are nothing to worry about.

Batching Log Updates

Imagine we create two files in a row called file1 and file2 in the same directory.

To create a file, one has to update: the inode bit map, the newly-created inode of the file, the data blocks of the parent directory containing the new directory.

entry, and the parent directory inode (which now has a new modification time).

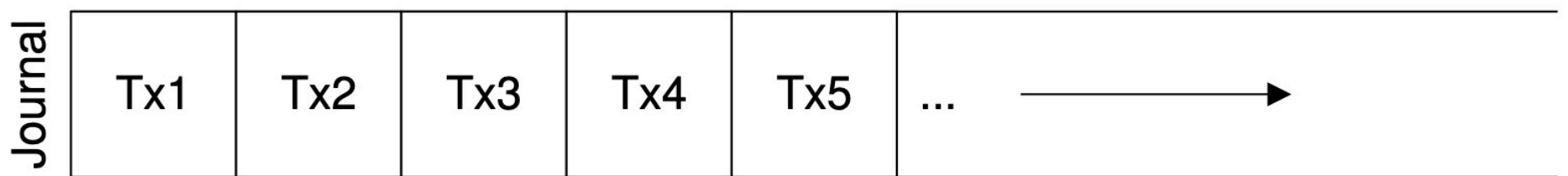
With journaling, we logically commit all of this information to the Journal for each of our two file creations; because the files are in the same directory, and assuming they have inodes within the same inode block, we end-up writing these same blocks twice.

To remedy this problem, some file systems don't commit each update to disk one at time. Instead, they buffer all updates into one global transaction.

After some time, this global transaction is committed.

Making The Log Finite

If we keep adding transactions to the log, it will soon fill up:



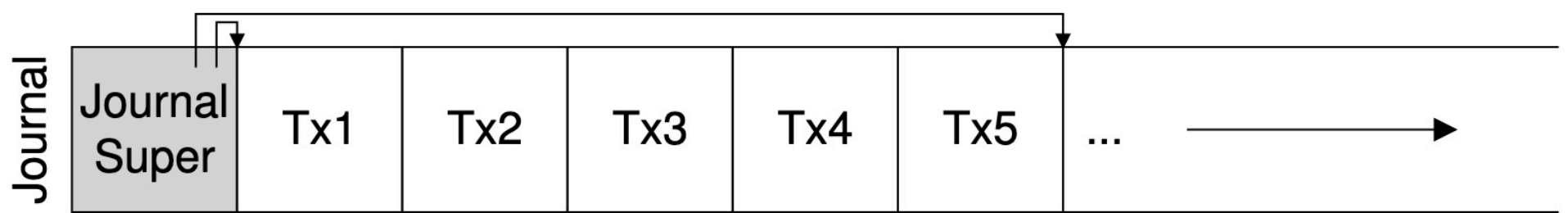
Two problems arise when the log becomes full:

1. The larger the log, the longer the recovery will take, as the recovery process must replay all the transactions within the log (in order) to recover.
2. No further transactions can be committed to the disk, thus making the file system useless.

To address this, file systems treat the log as a circular data structure, re-using it over & over; this is why a journal is called circular log.

Thus, the file system after a transaction has been checkpointed must free the space it was occupying within the journal.

To achieve this, you mark the oldest & newest non-checkpointed transactions in a journal superblock; all other space is free.



The Journaling superblock records enough information to know which transactions have not yet been checkpointed, and thus reduce recovery time as well as enables re-use of the log in a circular log fashion.

Thus, our updated protocol as:

1. **Journal write:** Write the contents of the transaction (containing TxB and the contents of the update) to the log; wait for these writes to complete.
2. **Journal commit:** Write the transaction commit block (containing TxE) to the log; wait for the write to complete; the transaction is now **committed**.
3. **Checkpoint:** Write the contents of the update to their final locations within the file system.
4. **Free:** Some time later, mark the transaction free in the journal by updating the journal superblock.

But we are still writing each data block to the disk twice, which is a heavy cost to pay.

Metadata Journaling

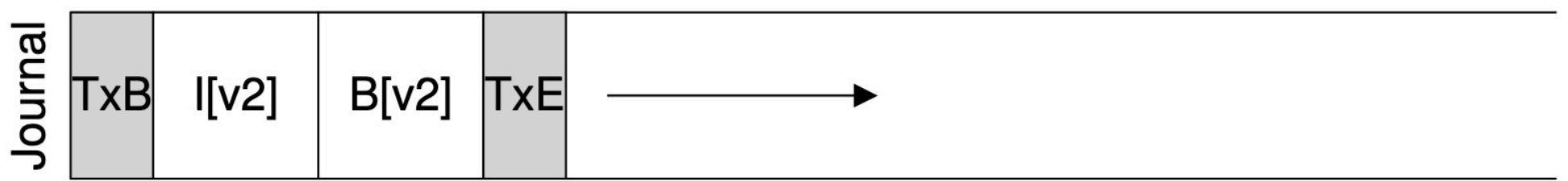
For each write to disk, we are also writing to the Journal first, thus doubling write traffic.

The mode of Journal described above is called data Journaling.

Because of its high cost, a simpler form of journaling called ordered journaling or metadata journaling.

It is same as data journaling except the user data block is not written to the Journal.

Thus, a transaction would look like:



The data block Db is written to the file system directly avoiding the extra write. As most I/O traffic is data blocks, this reduces the cost.

When should we write data blocks to disk?

Lets consider our above example append of a file. The update consists of three blocks: I[v2], B[v2] and Db.

Consider a case where I[v2] & B[v2] are written but Db did not make it to the disk. The file system will try to recover & produce a consistent file system (from perspective of file system metadata) but I[v2] will be pointing to garbage data.

To avoid this, file systems write data blocks to the disk first, before related metadata is written to the disk.

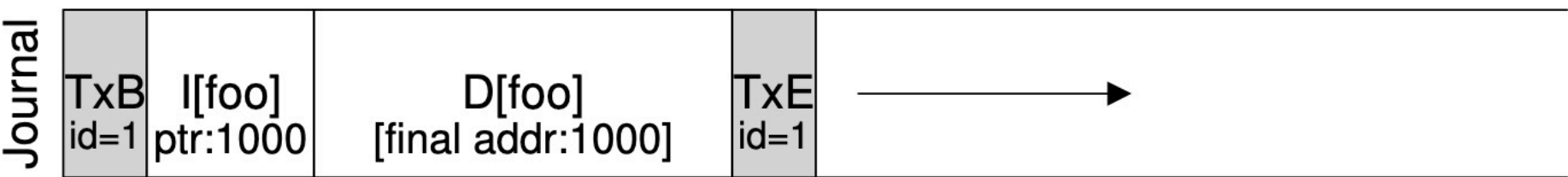
Thus, our updated protocol is:

1. **Data write:** Write data to final location; wait for completion (the wait is optional; see below for details).
2. **Journal metadata write:** Write the begin block and metadata to the log; wait for writes to complete.
3. **Journal commit:** Write the transaction commit block (containing TxE) to the log; wait for the write to complete; the transaction (including data) is now **committed**.
4. **Checkpoint metadata:** Write the contents of the metadata update to their final locations within the file system.
5. **Free:** Later, mark the transaction free in journal superblock.

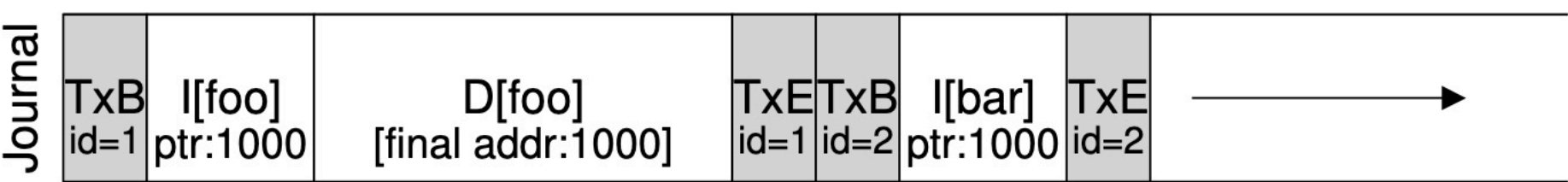
The only real requirement is that steps 1 & 2 complete before step 3.

Tricky Case: Block Reuse

The particular example Tweedie gives is as follows. Suppose you are using some form of metadata journaling (and thus data blocks for files are *not* journaled). Let's say you have a directory called `foo`. The user adds an entry to `foo` (say by creating a file), and thus the contents of `foo` (because directories are considered metadata) are written to the log; assume the location of the `foo` directory data is block 1000. The log thus contains something like this:



At this point, the user deletes everything in the directory and the directory itself, freeing up block 1000 for reuse. Finally, the user creates a new file (say `bar`), which ends up reusing the same block (1000) that used to belong to `foo`. The inode of `bar` is committed to disk, as is its data; note, however, because metadata journaling is in use, only the inode of `bar` is committed to the journal; the newly-written data in block 1000 in the file `bar` is *not* journaled.



Now assume a crash occurs and all of this information is still in the log. During replay, the recovery process simply replays everything in the log, including the write of directory data in block 1000; the replay thus overwrites the user data of current file `bar` with old directory contents! Clearly this is not a correct recovery action, and certainly it will be a surprise to the user when reading the file `bar`.

There are a number of solutions to this problem. One could, for example, never reuse blocks until the delete of said blocks is checkpointed out of the journal. What Linux ext3 does instead is to add a new type of record to the journal, known as a **revoke** record. In the case above, deleting the directory would cause a revoke record to be written to the journal. When replaying the journal, the system first scans for such revoke records; any such revoked data is never replayed, thus avoiding the problem mentioned above.

Wrapping Up Journaling: A Timeline

Before ending our discussion of journaling, we summarize the protocols we have discussed with timelines depicting each of them. Figure 42.1 shows the protocol when journaling data and metadata, whereas Figure 42.2 shows the protocol when journaling only metadata.

In each figure, time increases in the downward direction, and each row in the figure shows the logical time that a write can be issued or might complete. For example, in the data journaling protocol (Figure 42.1), the writes of the transaction begin block (TxB) and the contents of the transaction can logically be issued at the same time, and thus can be completed in any order; however, the write to the transaction end block (TxE) must not be issued until said previous writes complete. Similarly, the checkpointing writes to data and metadata blocks cannot begin until the transaction end block has committed. Horizontal dashed lines show where write-ordering requirements must be obeyed.

A similar timeline is shown for the metadata journaling protocol. Note that the data write can logically be issued at the same time as the writes to the transaction begin and the contents of the journal; however, it must be issued and complete before the transaction end has been issued.

Finally, note that the time of completion marked for each write in the timelines is arbitrary. In a real system, completion time is determined by the I/O subsystem, which may reorder writes to improve performance. The only guarantees about ordering that we have are those that must be enforced for protocol correctness (and are shown via the horizontal dashed lines in the figures).

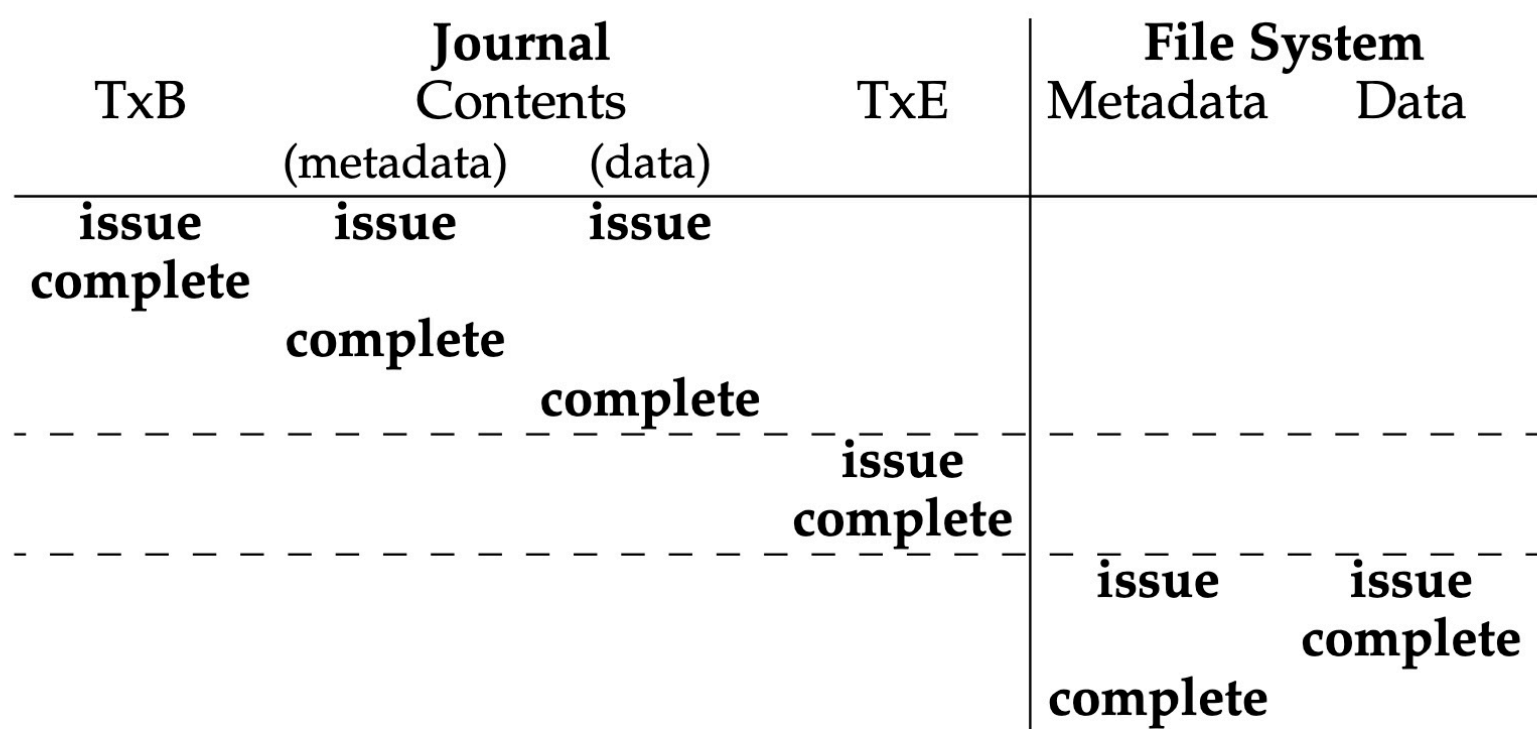
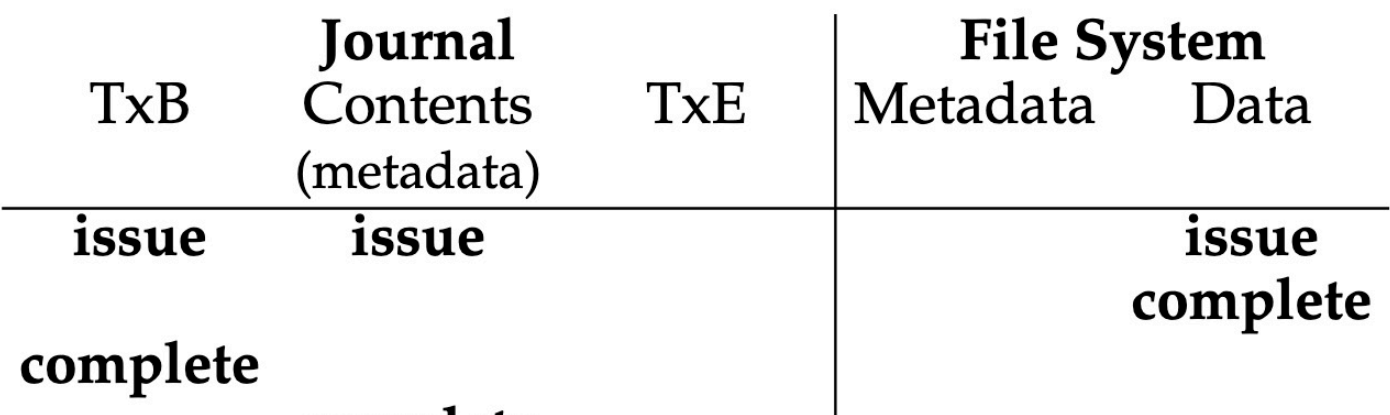


Figure 42.1: Data Journaling Timeline



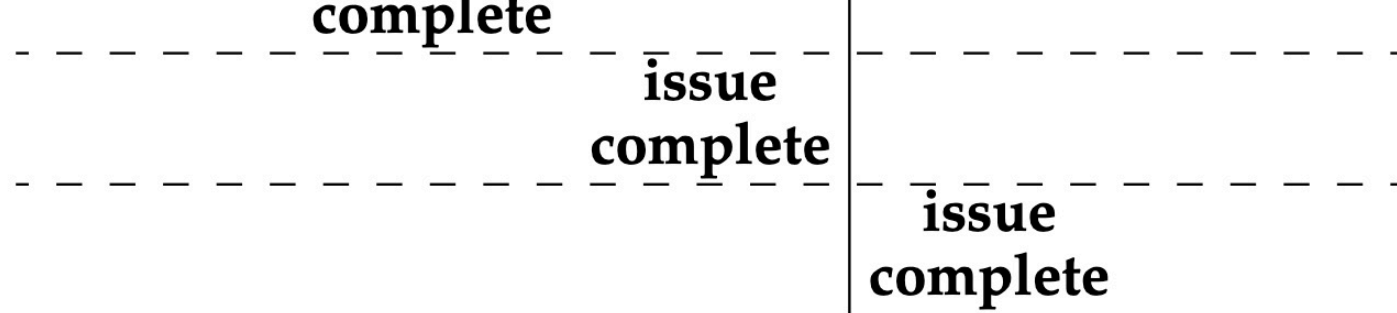


Figure 42.2: Metadata Journaling Timeline

Solution #3: Other Approaches

1. Soft updates:

This approach carefully orders all writes to the file system to ensure that the on-disk structures are never left in an inconsistent state.

For example, by writing a pointed-to-data block to disk before the inode that points to it, we can ensure that the inode never points to garbage.

Similar rules can be devised for all file system structures.

Implementing this requires intricate knowledge of each file system's data structure.

2. Copy-on-write (COW):

This technique never overwrites files or directories in place.

Instead, it places new updates to previously unused locations on disk. After a number of updates are completed, COW file systems flip the root structure of file system to include pointers to newly updated structures.

3. Backpointer-based Consistency (BBC):

To achieve consistency, an additional back pointer

is added to every block in the system.

For example, each data block has a reference to the inode to which it belongs.

When accessing a file, the file system can determine if the file is consistent by checking if forward pointer (address in inode or direct pointer) points to a block that refers to it.

If so, file is consistent; if not, the file is inconsistent.

4. Optimistic Crash Consistency:

This approach issues as many writes to disk as possible by using a generalized form of transaction checksum, and includes few other techniques to detect inconsistencies should they arise.

Summary:

Recovery time of fsck: $O(\text{size-of-disk-volume})$

Recovery time of Journaling: $O(\text{size-of-the-log})$